Back

REAL-TIME MULTI-USER CHAT APPLICATION PRO

Develop a client-server chat application that allows multiple users to join a chat room and communicate in real-time. This project will dive deep into networking, concurrency, and client-server architecture.

Python | Advanced | 86 tasks

Learning Outcomes

Socket programming for networking. | Client-server architecture. | Asynchronous I/O with `asyncio` for high concurrency. | Designing and implementing communication protocols. | Handling network streams and data serialization (e.g., JSON).

Start Project

Content About

1 Foundation & Protocol

Project Foundation and Communication Protocol Design

0 of 9 tasks completed 0%

- Create Project Root Directory
- Create a Python Virtual Environment
- Activate the Python Virtual Environment (venv)
- Create Client-Server Directory Structure
- Define the Base Communication Protocol
- Define the `join` Message Protocol
- Define the Public Message Protocol
- Define the Server-to-Client Broadcast Message
- Define the Server-to-Client `info` Message Type

2 Basic Server Skeleton

Implement a Basic Synchronous Server

0 of 13 tasks completed 0%

- Create the Main Server File `server.py`
- Import Python's Socket Library
- Define Server Host IP and Port in Python
- Create a TCP Server Socket
- Ensure Proper Socket Cleanup with try...finally
- Bind a Python Socket to a Network Address
- Enable Server Socket to Listen for Connections
- Python Server: Accepting Client Connections
- Receive Data from a Client with conn.recv()
- Decode Received Bytes into a String
- Display Client Message on the Server Console
- Sending a Response from a Python Socket Server
- Gracefully Close the Client Connection in a Python Server

3 Basic Client Skeleton

Implement a Basic Synchronous Client

0 of 11 tasks completed 0%

- Create the Client Entry Point: client.py
- Import Socket Library for Python Client
- Define the Server Address in the Client Script
- Create a Client Socket Object in Python
- Connect Python Client Socket to a Server
- Capture User Input in Python Client
- Encode a String to Bytes for Network Transmission
- Transmit Data from Client using `sendall()`
- Receive Server Response using `socket.recv()`
- Decode and Display the Server's Response
- Gracefully Closing the Client Socket in Python

4 Asynchronous Server Core

Refactor the Server for Concurrency with `asyncio`

0 of 11 tasks completed 0%

- Begin Asynchronous Server Refactor with asyncio
- Introduce an asyncio Coroutine for Client Handling
- Identify Client in Asyncio Server using `get_extra_info`
- Read Data Asynchronously with `await reader.read()`
- Send an Asynchronous Reply with writer.write() and writer.drain()
- Implement a Persistent Connection Loop for the Chat Server
- Implement Graceful Connection Cleanup in Asyncio Server
- Refactor to an `async def main()` Entry Point
- Initialize the Asynchronous Server with `asyncio.start_server`
- Run the Asyncio Server Indefinitely
- Create the Entry Point for the Asyncio Server

5 Multi-User Chat Logic

Implement Server-side State Management and Message Broadcasting

0 of 10 tasks completed 0%

- Implement Server-Side Client State
- Implement an Asyncio Broadcast Function for a Chat Server
- Implement a Robust Broadcast Function in an Asyncio Chat Server
- Implement Client Join and Nickname Validation Protocol
- Register a New Client in the Chat Server
- Implement the Main Chat Message Loop
- Implement Chat Message Broadcasting
- Implement Structured Cleanup for Client Disconnections
- Implement User Disconnection Cleanup
- Broadcast User Departure Message

6 Interactive Async Client

Enhance the Client for Asynchronous I/O

0 of 10 tasks completed 0%

- Enable Asynchronous Operations in the Chat Client
- Implement Asynchronous Client Connection
- Create an Asynchronous Message Listener for a Chat Client
- Create an Asynchronous Client-Side Message Listener
- Define the Client's `send_messages` Coroutine
- Implement Client Message Sending with Asyncio
- Integrate Blocking I/O with asyncio.to_thread
- Implement Client Join Handshake
- Run Chat Client Coroutines Concurrently with asyncio.gather
- Implement Robust Error Handling and Graceful Shutdown in an Asyncio Client

7 Feature: Chat Rooms

Enhancement: Implement Multiple Chat Rooms

0 of 8 tasks completed 0%

- Update Chat Protocol for Multi-Room Support
- Refactor Server State for Multi-Room Chat Support
- Refactor Server to Handle Multi-Room User Joins
- Implement Targeted Broadcasting for a Multi-Room Chat Server
- Implement a `/join` Command for a Python Chat Server
- Implement /list_users Command in Chat Server
- Implement a /list_rooms Command for a Chat Server
- Implement Robust Client Disconnect and Cleanup Logic

8 Feature: Private Messages

Enhancement: Implement Private Messaging

0 of 7 tasks completed 0%

- Update Chat Protocol for Private Messaging
- Implement Client-Side Whisper Command
- Implement Recipient Search for Private Messages
- Refactor Private Message Logic to Retrieve Recipient's Writer
- Implement Unicast Private Message Delivery
- Implement Sender Confirmation for Private Messages
- Implement Private Message Error Handling

9 Bonus: Message History

Portfolio Enhancement: Persistent Chat History

0 of 7 tasks completed 0%

- Install aiosqlite for Asynchronous Database Access
- Design a SQLite Schema for a Chat Messages Table
- Initialize SQLite Database for Chat History
- Save Chat Messages to a SQLite Database
- Create a Function to Fetch Recent Chat Messages from Database
- Unicast Chat History to New Clients
- Review Asynchronous Database Connection Strategy